

中山大學
SUN YAT-SEN UNIVERSITY



国家超级计算广州中心
NATIONAL SUPERCOMPUTER CENTER IN GUANGZHOU

CUDA编程模型

任课教师：吴迪

Roadmap

- **Introducing the CUDA Programming Model**
- **Timing Your Kernel**
- **Organizing Parallel Threads**
- **Managing Devices**

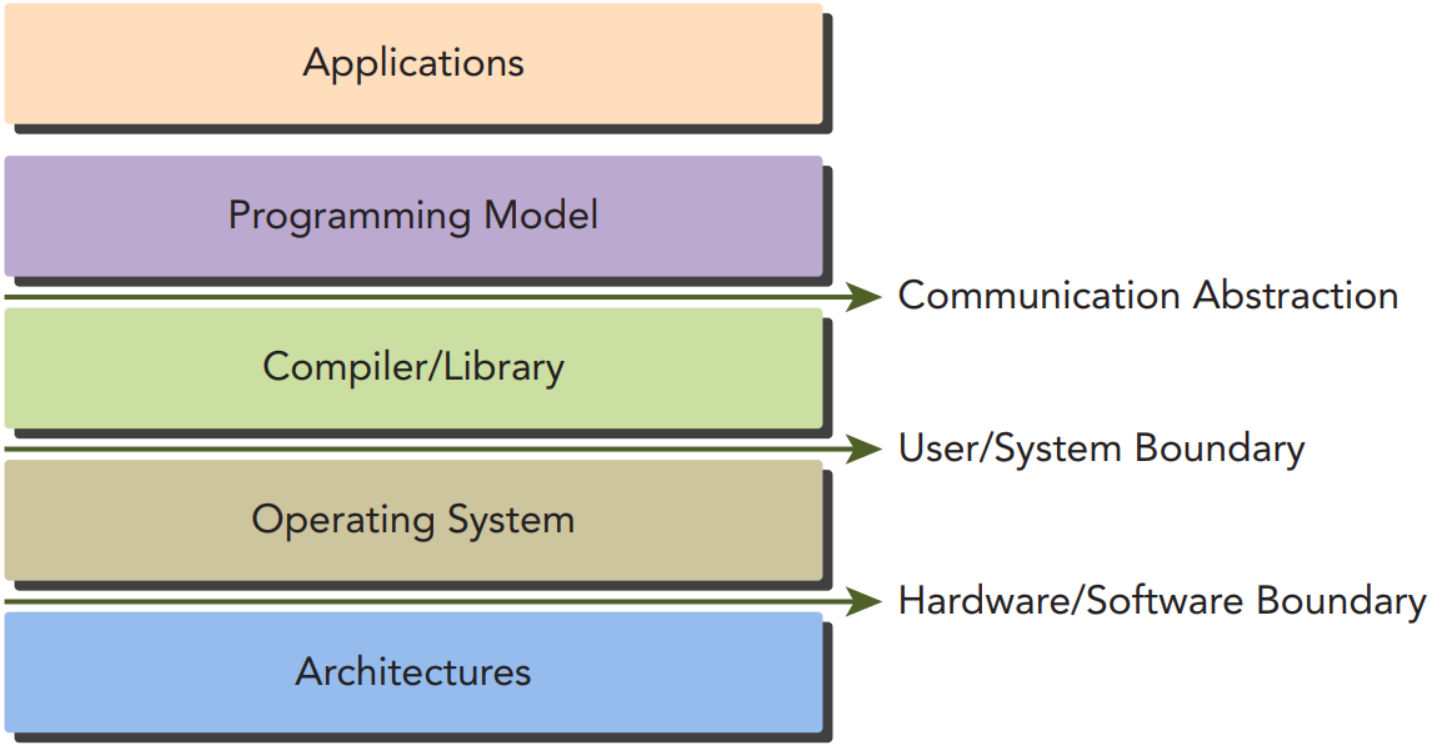
参考阅读：Cheng-Ch2

1. Introducing the CUDA Programming Model

1.1 CUDA Programming Structure

Programming Model

- Programming models present an abstraction of computer architectures
  - act as a bridge between an application and its implementation on available hardware.



Programming Model

- **The program**, written for a programming model, dictates how **components** of the program share information and coordinate their activities.

- The programming model provides a **logical view** of specific computing architectures.

- Typically, it is **embodied** in a programming language or programming environment.

Levels of Parallel Computation

- From the perspective of a programmer, you can view **parallel computation** from different levels:
  - **Domain level:** how to decompose data and functions so as to solve the problem
  - **Logic level:** how to organize your concurrent threads
  - **Hardware level:** how threads are mapped to cores may help improve performance

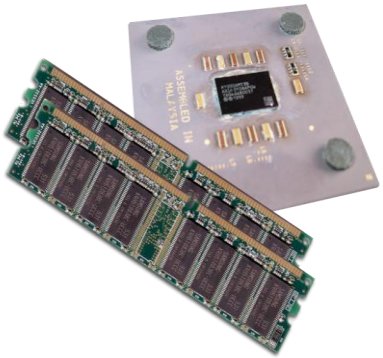
CUDA Programming Model

- CUDA programming model provides：
  - A way to organize threads on the GPU through a hierarchy structure
  - A way to access memory on the GPU through a hierarchy structure

- CUDA programming model enables you to execute applications on heterogeneous computing systems
  - by simply annotating code with a small set of extensions to the C programming language.

Terminology

- ***Host*** The **CPU** and its memory (host memory)
- ***Device*** The **GPU** and its memory (device memory)



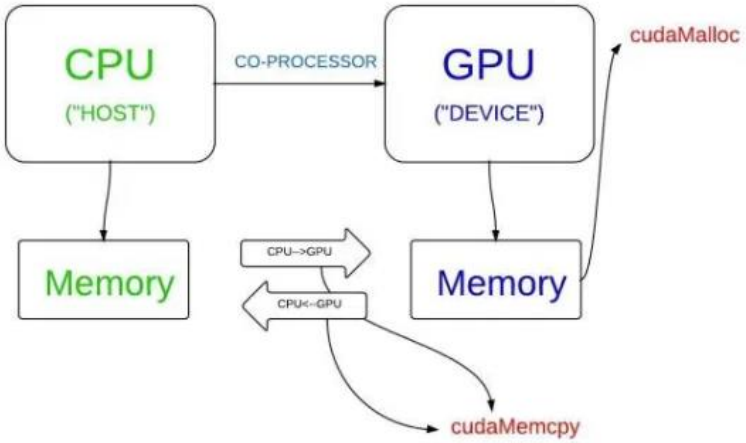
Host



Device

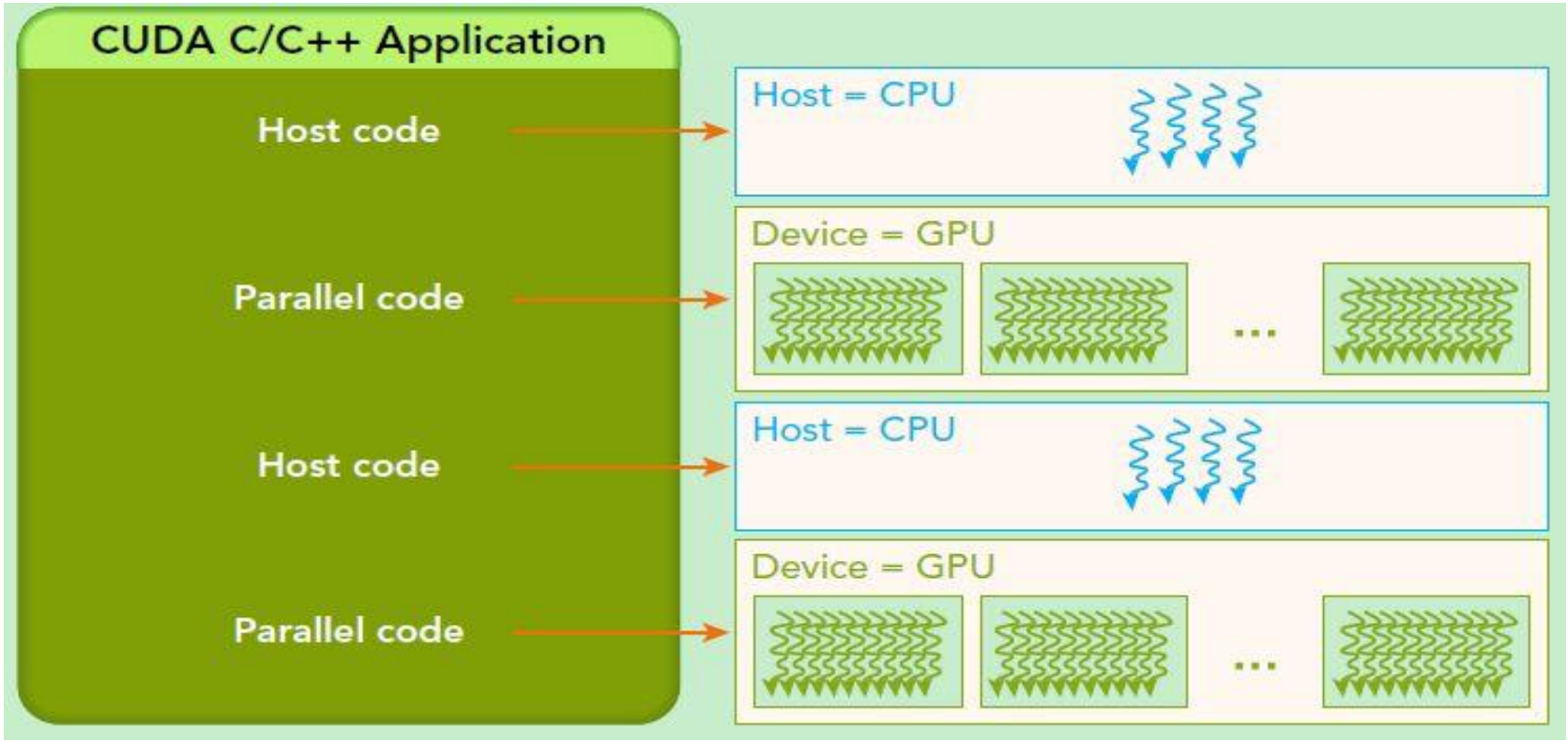
CUDA Programming Model

- In CUDA, **GPU** is viewed as ***compute device*** capable of executing a very high number of threads in parallel
- GPU operates as a coprocessor to the main **CPU (called *host*)**
- Compute-intensive portions of applications running on the host are **off-loaded** onto the device.



CUDA Program

- CUDA program: **host code + device code**



CUDA Kernel

- **Kernel**: the code that runs on the GPU device.
  - As the developer, you can express a kernel as a sequential program.

- CUDA manages scheduling programmer-written kernels on GPU threads.

- From the host, you define how your algorithm is mapped to the device based on application data and GPU device capability.

CUDA Kernel

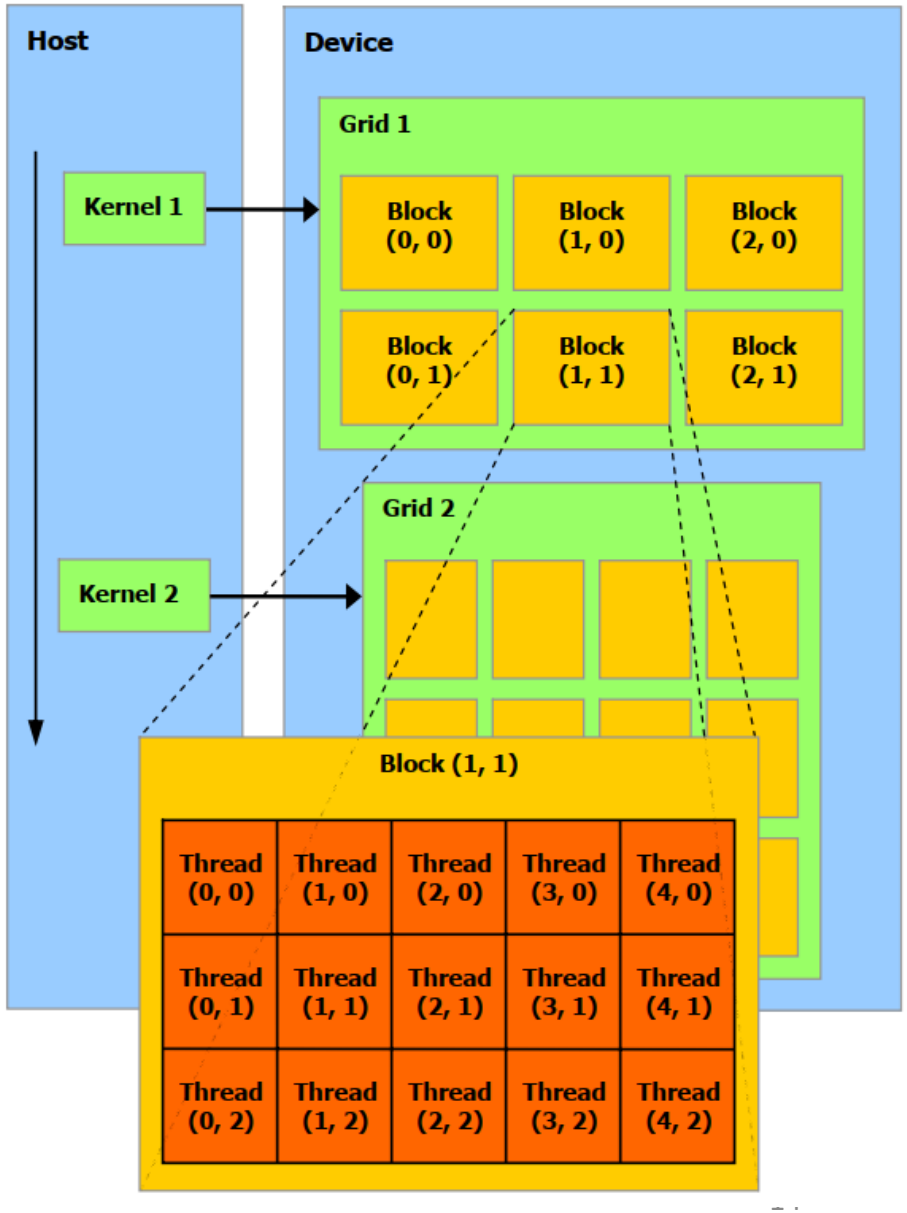
- The parallel portion of an application can be isolated into a ***function*** that is executed on the device as many different ***threads***.

- Such a ***function*** is compiled to **the instruction set of the device** and the ***resulting program***, called a ***kernel***

- The batch of threads that executes a kernel is organized as a **grid** of thread blocks

Thread Batching

- The host issues a succession of kernel invocations to the device.
- Each kernel is executed as a batch of threads organized as a grid of thread blocks

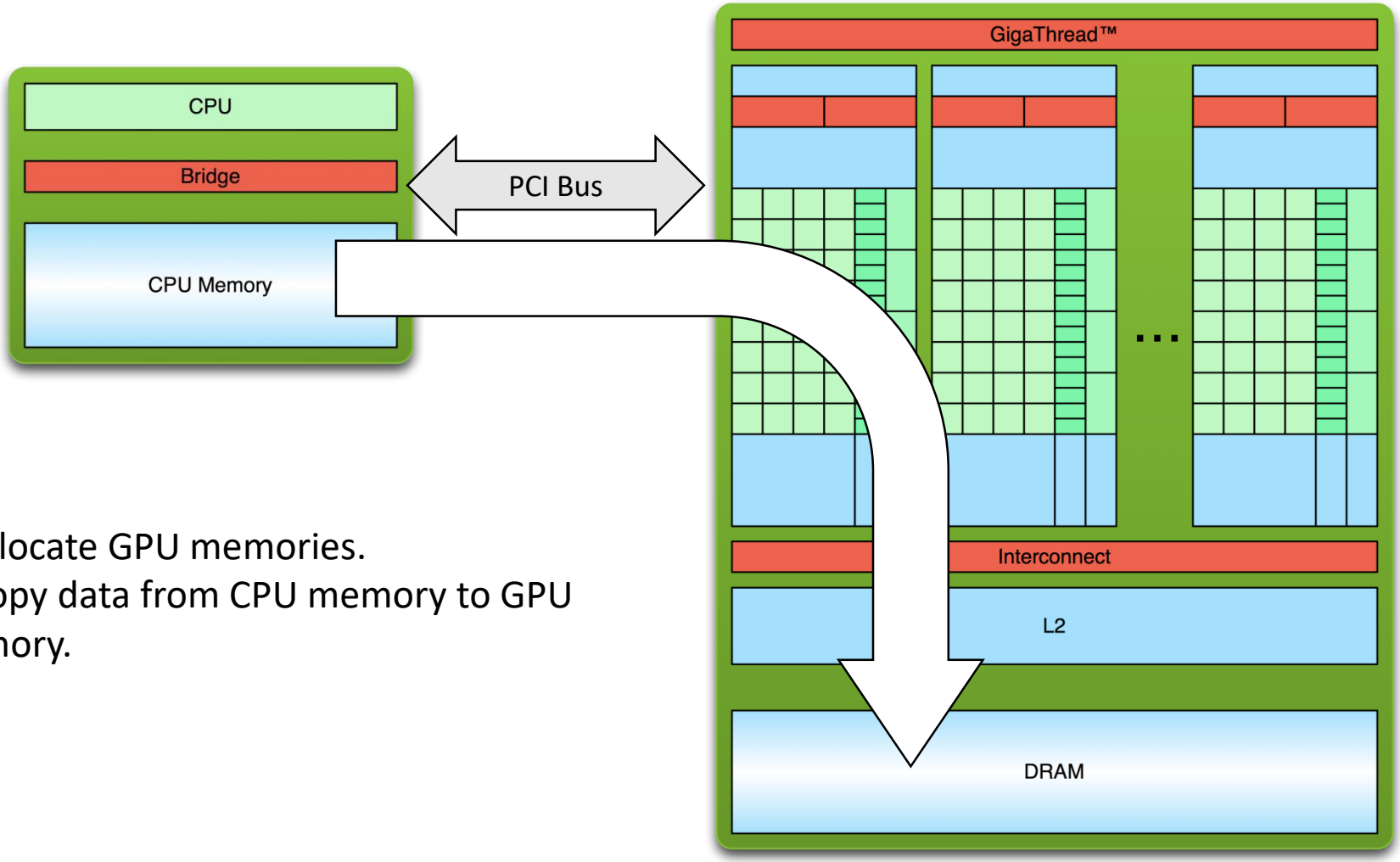


CUDA Program Structure

A typical CUDA program structure consists of **5 steps**:

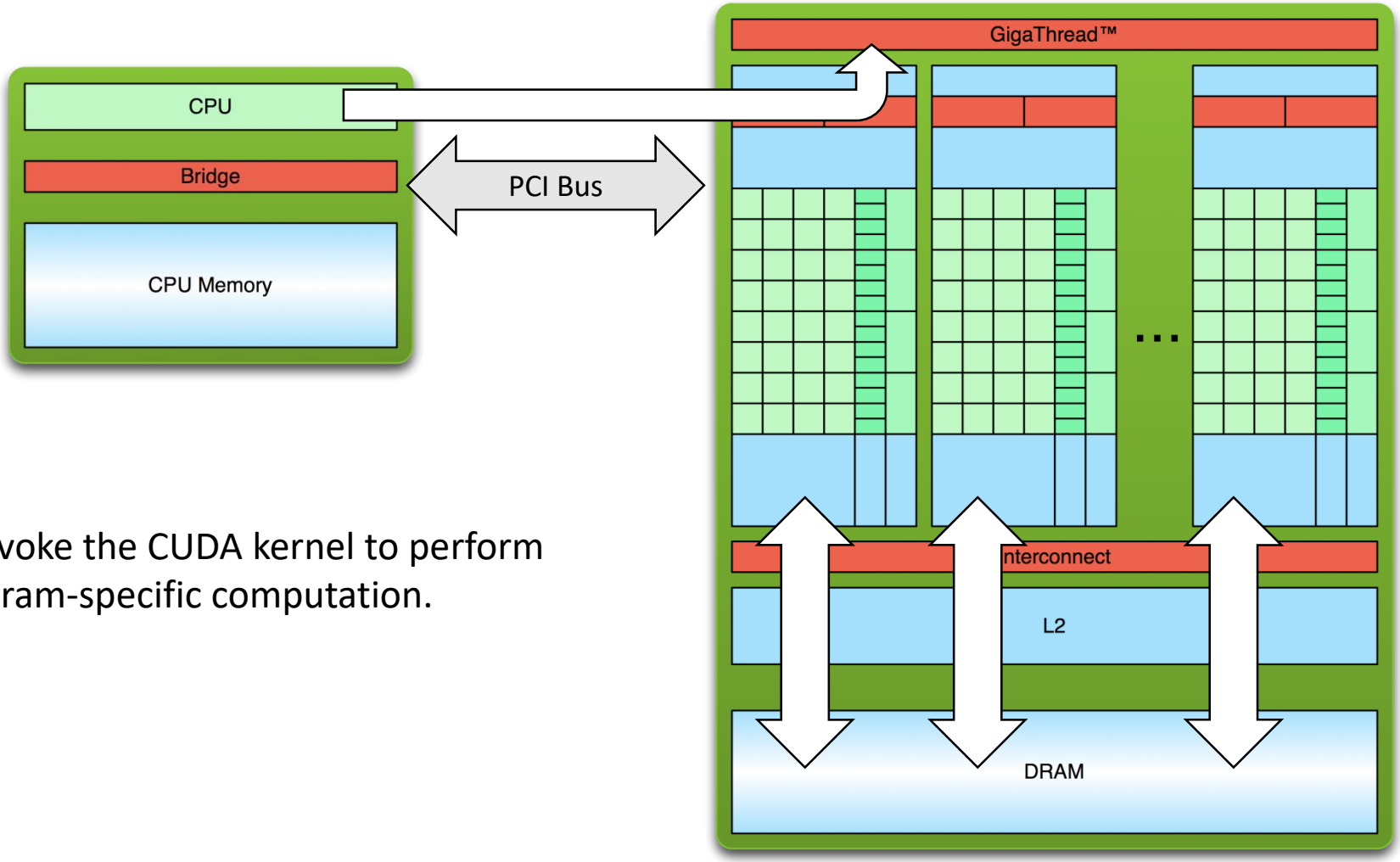
1. Allocate GPU memories.
2. Copy data from CPU memory to GPU memory.
3. Invoke the CUDA kernel to perform program-specific computation.
4. Copy data back from GPU memory to CPU memory.
5. Destroy GPU memories.

Simple Processing Flow



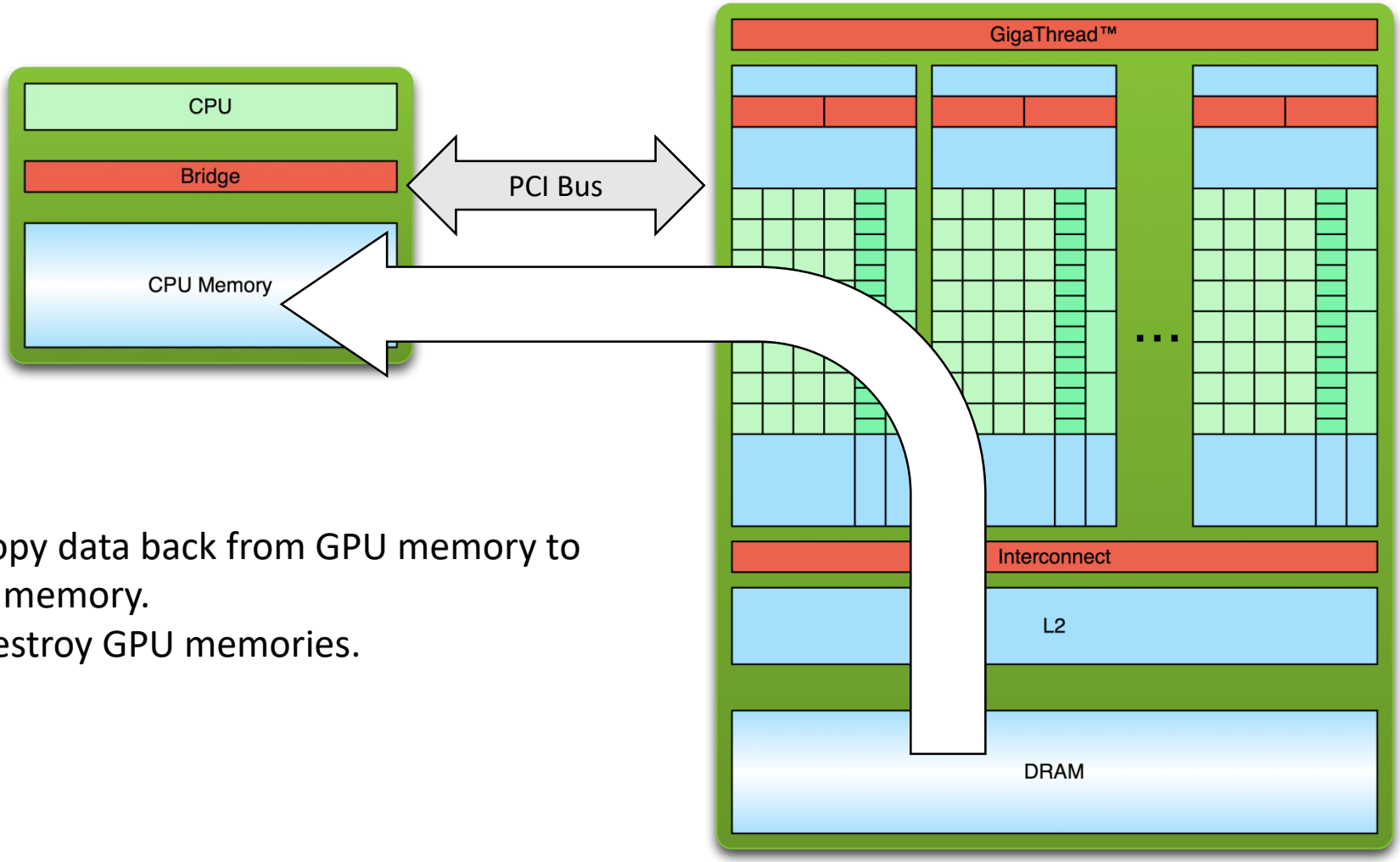
1. Allocate GPU memories.
2. Copy data from CPU memory to GPU memory.

Simple Processing Flow



3. Invoke the CUDA kernel to perform program-specific computation.

Simple Processing Flow



4. Copy data back from GPU memory to CPU memory.
5. Destroy GPU memories.

1.2 Managing Memory

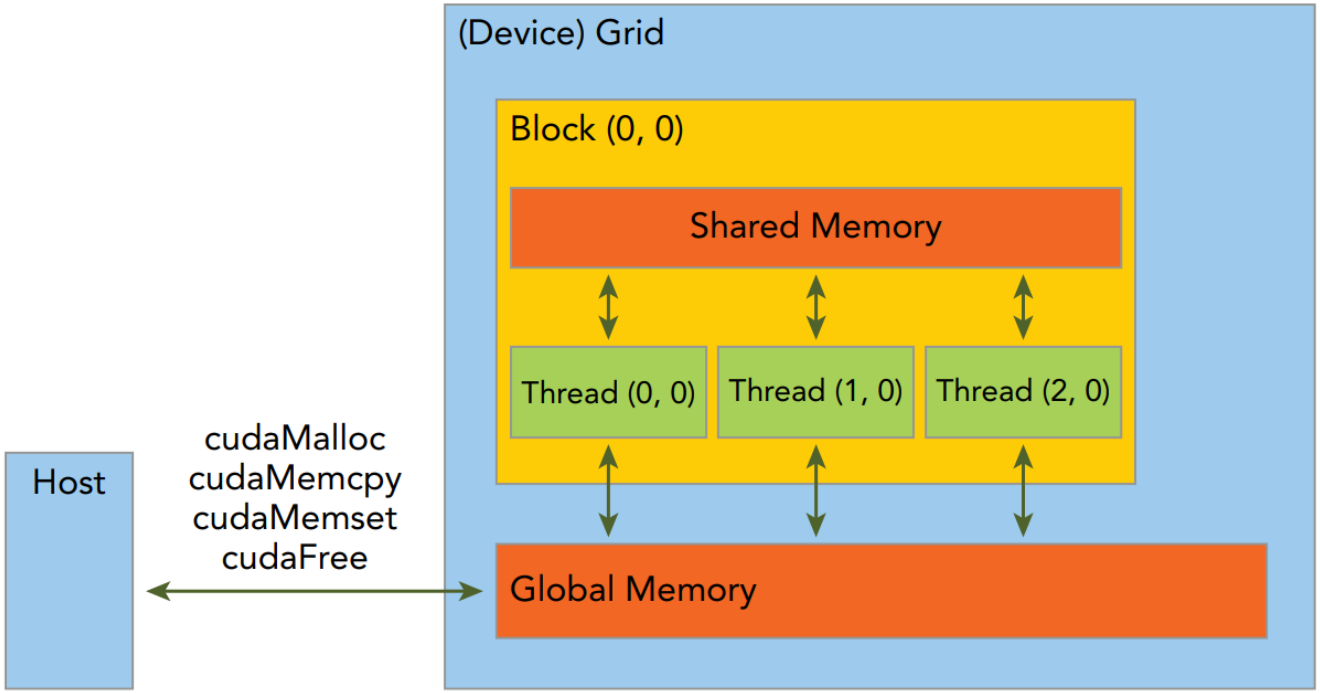
Managing Memory

- The CUDA programming model assumes a system composed of a host and a device, each with its own separate memory.
- CUDA runtime provides a set of memory management functions

STANDARD C FUNCTIONS	CUDA C FUNCTIONS
`malloc`	`cudaMalloc`
`memcpy`	`cudaMemcpy`
`memset`	`cudaMemset`
`free`	`cudaFree`

GPU Memory Hierarchy

- One of the more notable characteristics of CUDA is the exposed memory hierarchy
  - Each GPU device has a set of different memory types used for different purposes.



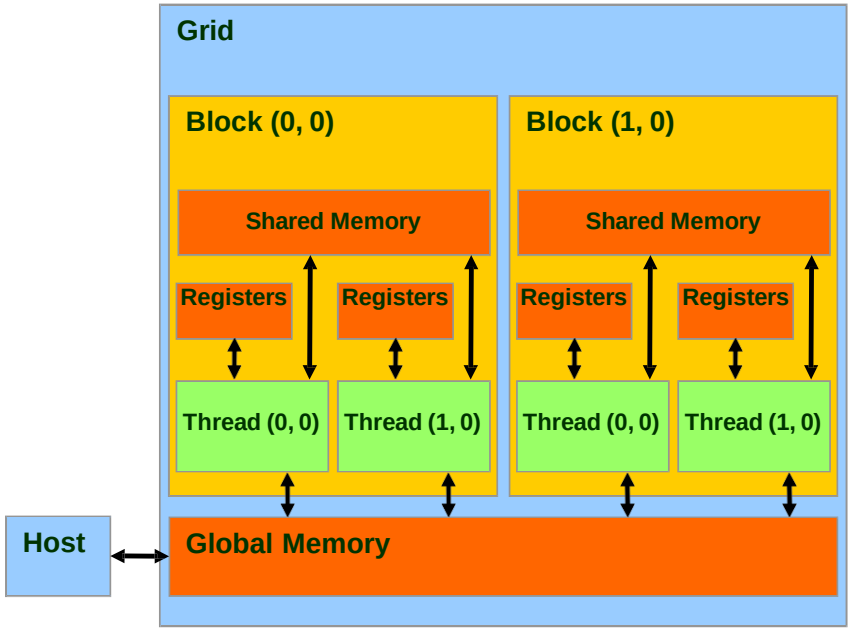
CUDA Memory Model

Global memory

- Main means of communicating R/W data between **host** and **device**
- Contents visible to all threads
- Long latency access

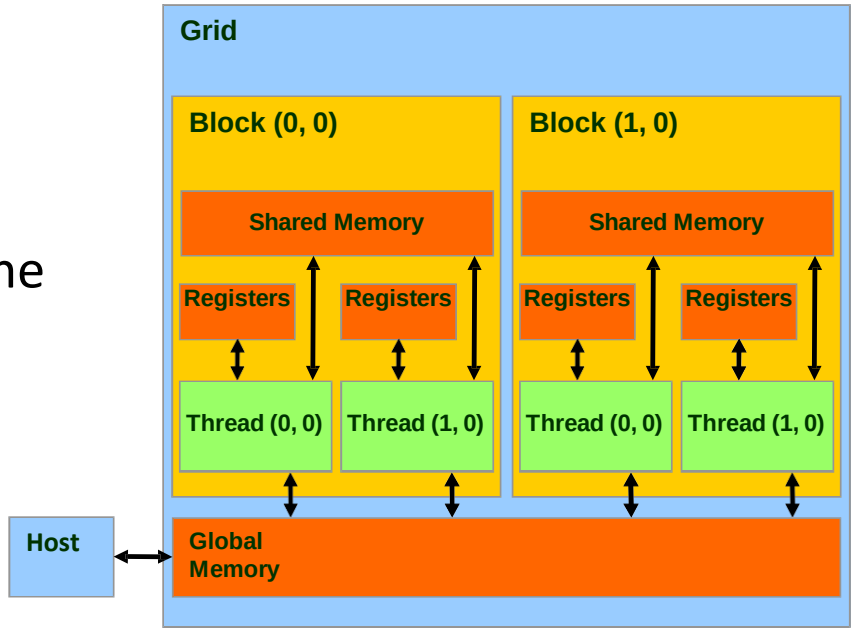
Device code can:

- R/W per-thread registers
- R/W per-grid global memory



CUDA Device Memory Allocation

- **`cudaMalloc()`**
  - Allocates object in the device **Global Memory**
  - Requires two parameters
    - **Address of a pointer** to the allocated object
    - **Size of allocated object**
- **`cudaFree()`**
  - Free object from device Global Memory
    - Pointer to freed object

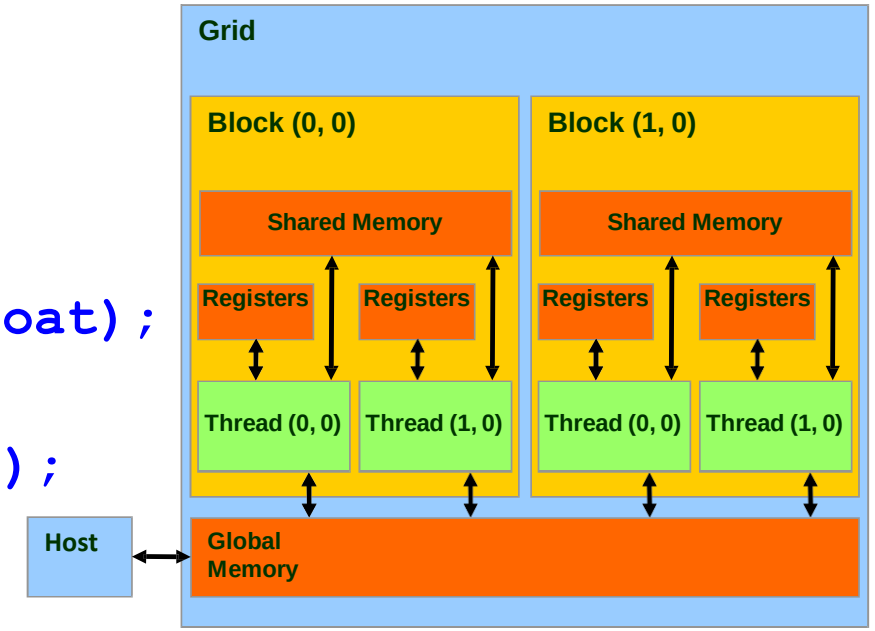


CUDA Device Memory Allocation

Example:

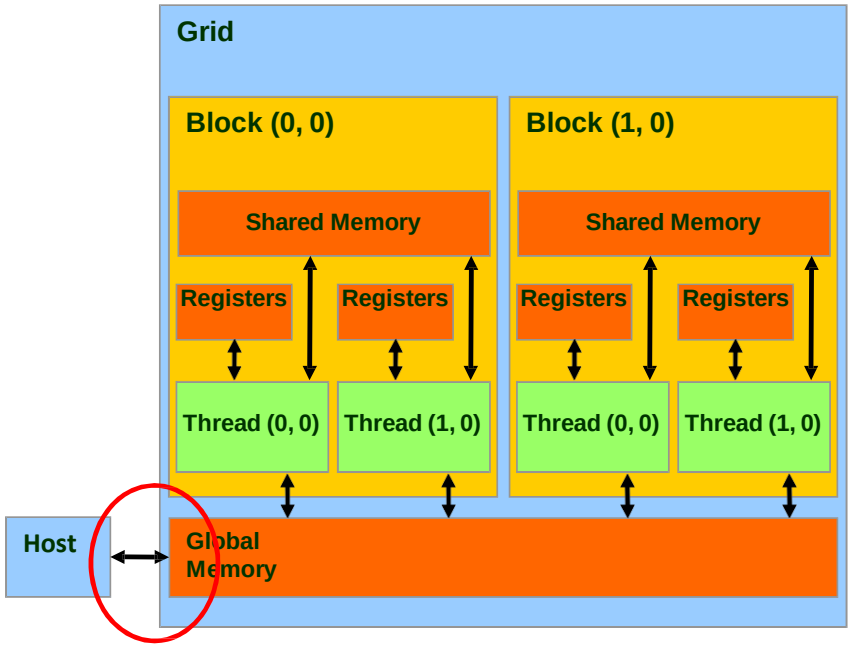
```
WIDTH = 64;
float *      Md;
int size = WIDTH * sizeof(float);

cudaMalloc((void**)&Md, size);
cudaFree(Md);
```



CUDA Device Memory Allocation

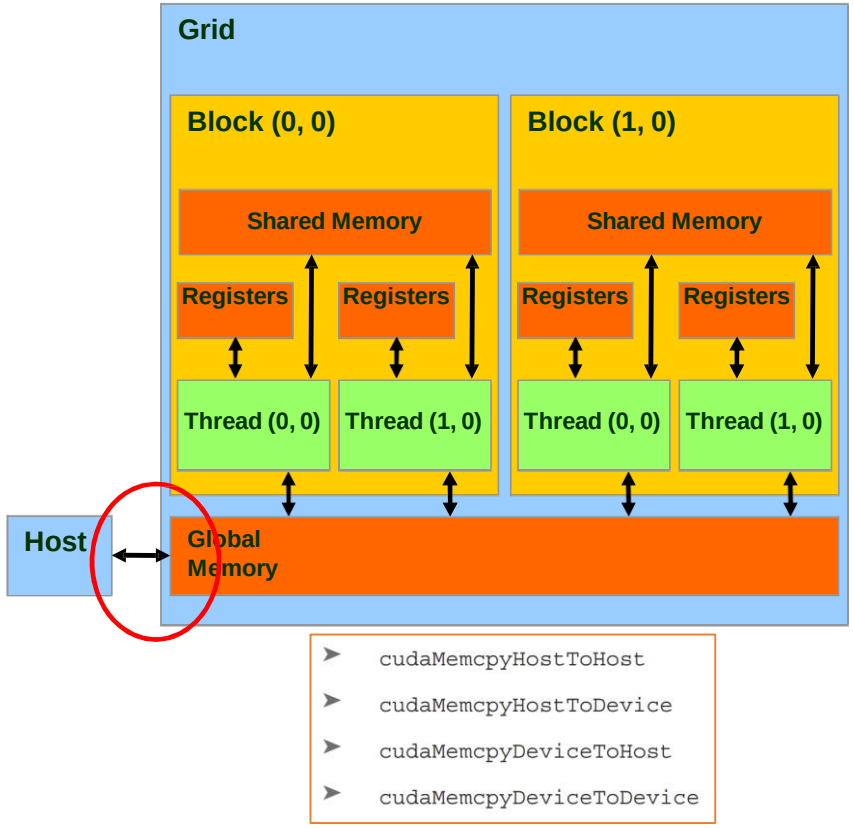
- `cudaMemcpy()`
  - memory data transfer
  - Requires four parameters
    - Pointer to destination
    - Pointer to source
    - Number of bytes copied
    - Type of transfer
      - Host to Host
      - Host to Device
      - Device to Host
      - Device to Device
- Asynchronous transfer



CUDA Device Memory Allocation

Example:

Destination pointer **Source pointer** **Size in bytes** **Direction**

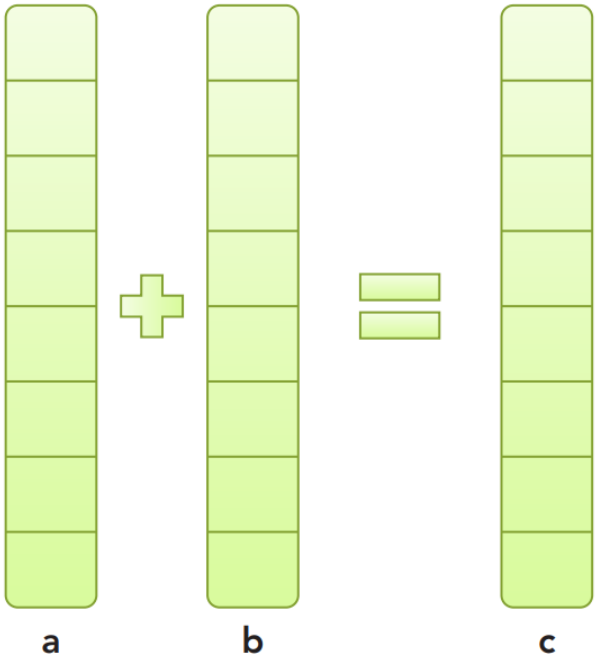



```
cudaMemcpy(Md, M, size, cudaMemcpyHostToDevice);

cudaMemcpy(M, Md, size, cudaMemcpyDeviceToHost);
```

Example: Array summation

- Program with CUDA C through a simple example of summing two arrays



Host-based array summation (sumArraysOnHost.c)

```
void sumArraysOnHost(float *A, float
*B, float *C, const int N) {
  for (int idx=0; idx<N; idx++) {
     C[idx] = A[idx] + B[idx];
  }
}
```

Example: Array summation

- modify the code to perform array summation on the GPU

```
float *d_A, *d_B, *d_C;
cudaMalloc((float**)&d_A, nBytes);
cudaMalloc((float**)&d_B, nBytes);
cudaMalloc((float**)&d_C, nBytes);

cudaMemcpy(d_A, h_A, nBytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, nBytes, cudaMemcpyHostToDevice);

//the kernel function can be invoked from the host
side to perform the array summation on the GPU

cudaMemcpy(gpuRef, d_C, nBytes,
    cudaMemcpyDeviceToHost);

cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
```

Use cudaMalloc to allocate the memory on the GPU.

transfer the data from the host memory to GPU global memory.

Introduced later

Copy the result from the GPU memory back to the host array gpuRef using cudaMemcpy

use cudaFree to release the memory used on the GPU.

Different Memory Spaces

- One of the most common **mistakes** in CUDA C is to improperly dereference different memory spaces.

- For the memory allocated on the GPU, the device pointers may not be dereferenced in the host code.

```
gpuRef = d_C    //the application will crash at runtime
```



```
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost)
```

Pointer in the host code

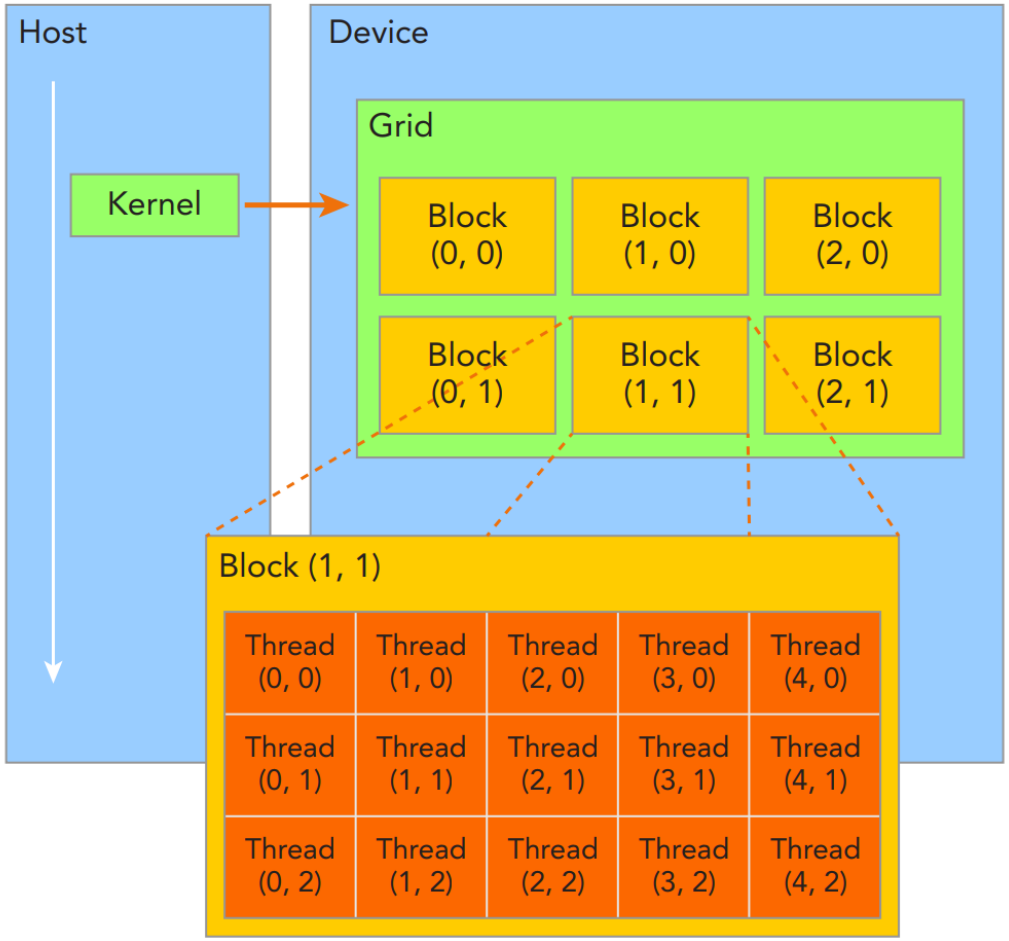
Device pointer

Note: Dereferencing （解引用） is a method used to manipulate or access data contained in a memory location that it is pointed to by a pointer.

1.3 Organizing Threads

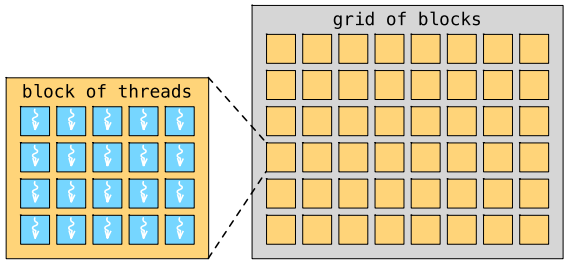
Organizing Threads

- CUDA exposes a **thread hierarchy** abstraction to organize your threads.



This is a two-level thread hierarchy decomposed into blocks of threads and grids of blocks

Grid and Block

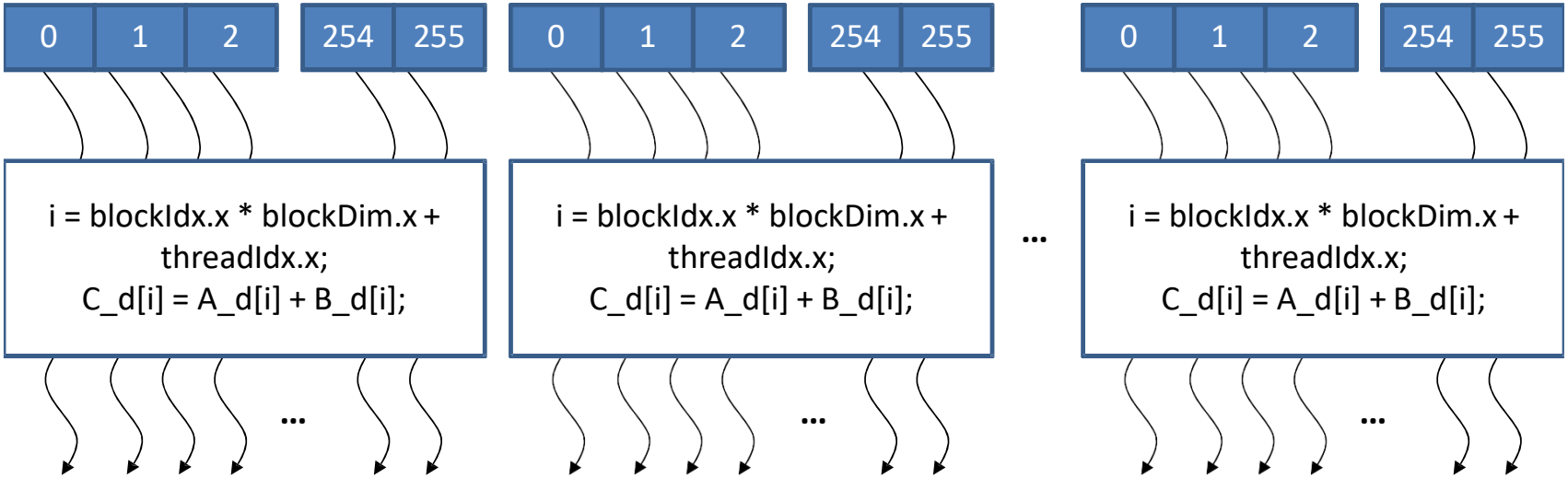


- **Grid**:
  - All threads spawned by a single kernel launch are collectively called a grid.
  - All threads in a grid share the same global memory space.
  - A grid is made up of many thread blocks.

- **Thread Block**:
  - A thread block is a group of threads that can cooperate with each other.
  - Threads from different blocks cannot cooperate.
  - Threads rely on **`blockIdx`** and **`threadIdx`** to distinguish themselves from each other:
    - **blockIdx (block index within a grid)**
    - **threadIdx (thread index within a block)**

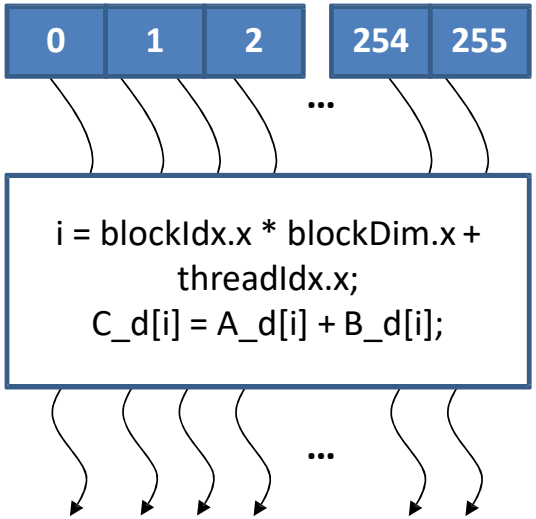
Grid

- **Grid**: a set of blocks with same dimensionality and size that execute the same kernel can be **batched together** into a **grid**



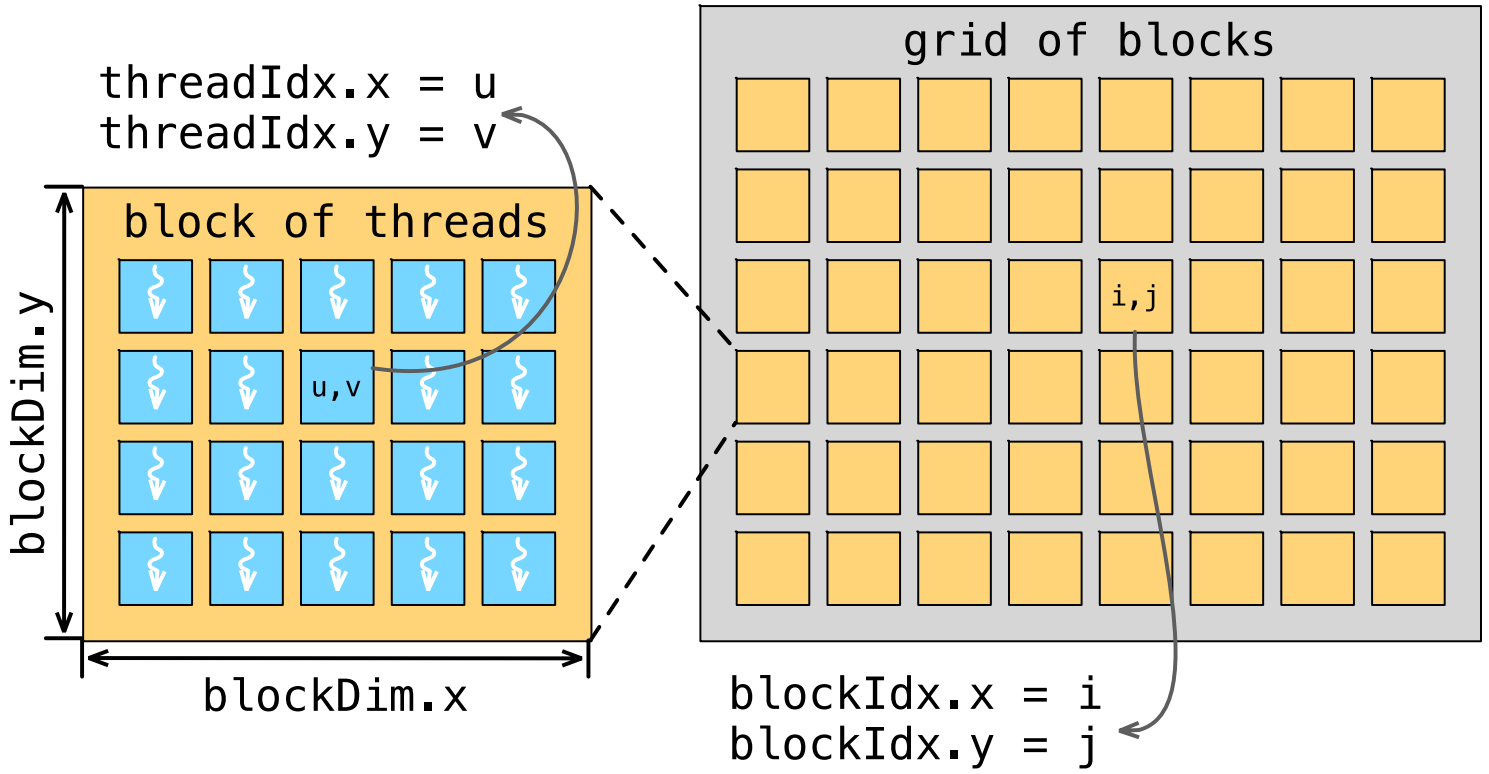
Block

- **Block**: a batch of threads that can cooperate together by sharing data and synchronizing their execution.
  - All threads run the same code (the SP in SPMD)
  - Each thread has an ID that it uses to compute memory addresses and make control decisions



blockIdx and threadIdx

- **Built-in, pre-initialized variables** that can be accessed within kernel functions.
  - When a kernel function is executed, blockIdx and threadIdx are assigned to each thread by the CUDA runtime.



blockIdx and threadIdx

- The coordinate variable is of type uint3, a CUDA built-in vector type.
  - a structure containing three unsigned integers
  - accessible through its x, y, and z fields

```
blockIdx.x
blockIdx.y
blockIdx.z
```

```
threadIdx.x
threadIdx.y
threadIdx.z
```

gridDim and blockDim

```
kernel_routine<<<gridDim, blockDim>>>(args);
```

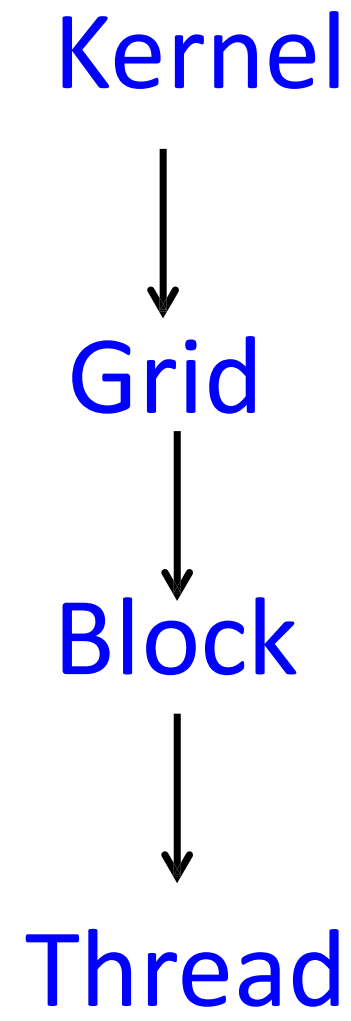
- **A collection of blocks form a grid (1D, 2D or 3D)**
  - Built-in variable gridDim specifies the size (or dimension) of the grid.
  - Each copy of the kernel can determine which block it is executing with the built-in variable blockIdx
- **Threads in a block are arranged in 1D, 2D, or 3D arrays**
  - Built-in variable blockDim specifies the size (or dimensions) of block.
  - threadIdx index (or 2D/3D indices) thread within a block

gridDim and blockDim

- The dimensions of a grid and a block are specified by:
  - blockDim (block dimension, measured in threads)
  - gridDim (grid dimension, measured in blocks)
- These variables are of type dim3, an integer vector type based on uint3
  - Both grids and blocks use the dim3 type with three unsigned integer fields.
  - The unused fields will be initialized to 1 and ignored.

```
blockDim.x        gridDim.x
blockDim.y        gridDim.y
blockDim.z        gridDim.z
```

Usually, a grid is organized as a 2D array of blocks, and a block is organized as a 3D array of thread

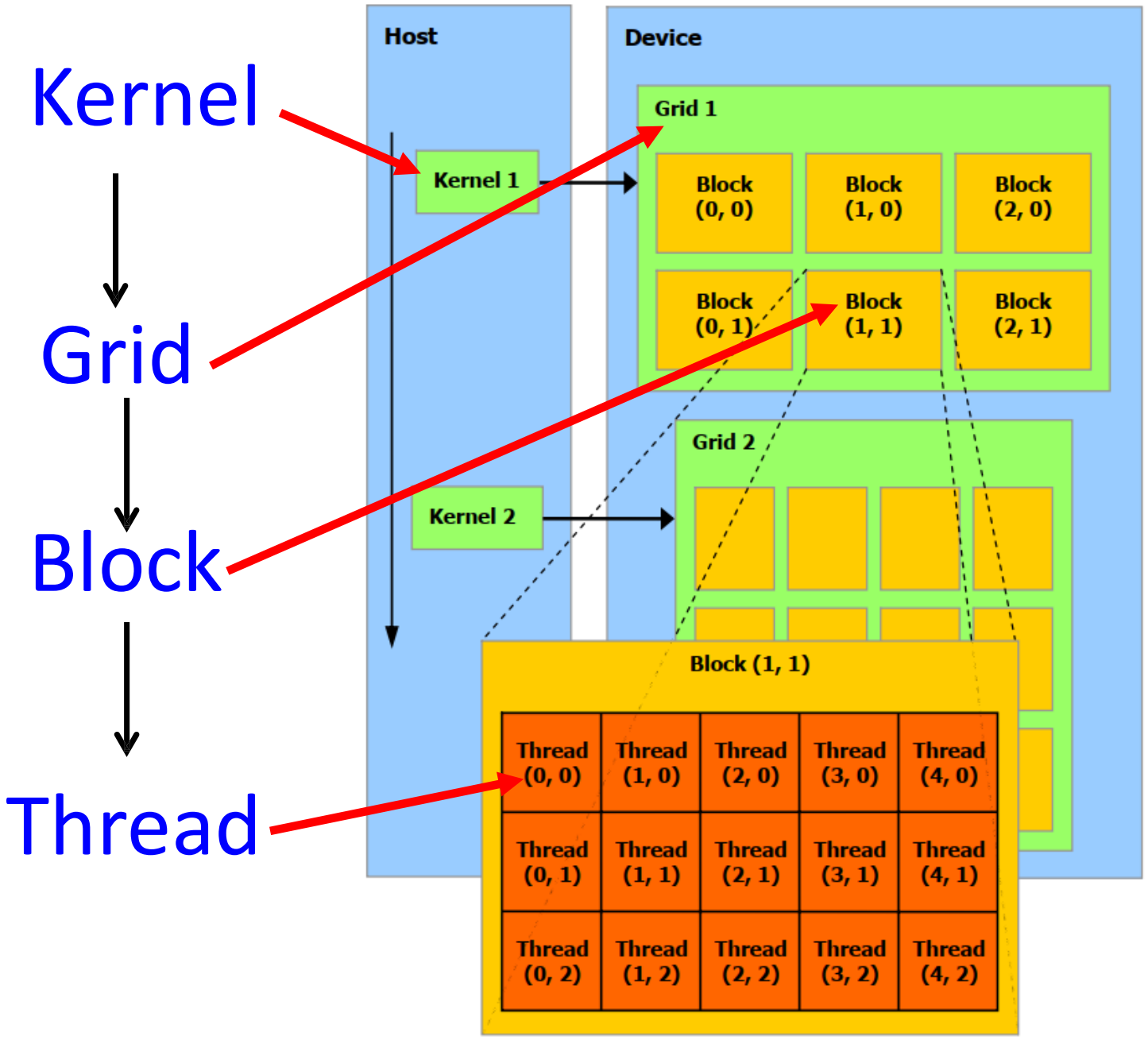


- Launched by the host
- Very similar to a C function
- To be executed on device
- All threads will execute that same code in the kernel.

- 1D or 2D (or 3D) organization of a block
- blockDim.x and blockDim.y
- gridDim.x and gridDim.y

- 1D, 2D, or 3D organization of a block
- Block is assigned to an SM
- blockIdx.x, blockIdx.y, and blockIdx.z

- threadIdx.x, threadIdx.y, and threadIdx.z


Kernel
Grid
Block
Thread
Host
Device
Kernel 1
Kernel 2
Grid 1
Block (0, 0)
Block (1, 0)
Block (2, 0)
Block (0, 1)
Block (1, 1)
Block (2, 1)
Grid 2
Block (1, 1)
Thread (0, 0)
Thread (1, 0)
Thread (2, 0)
Thread (3, 0)
Thread (4, 0)
Thread (0, 1)
Thread (1, 1)
Thread (2, 1)
Thread (3, 1)
Thread (4, 1)
Thread (0, 2)
Thread (1, 2)
Thread (2, 2)
Thread (3, 2)
Thread (4, 2)

Check Grid and Block Indices

- Check grid and block indices and dimensions (checkDimension.cu)

- define the block size and calculate the grid size based on the block and data size.
- the grid size is rounded up to the multiple of the block size

```
int nElem = 6;

dim3 block(3);
dim3 grid((nElem+block.x-1)/block.x);

printf("grid.x %d grid.y %d grid.z %d\n",grid.x, grid.y,
grid.z);
printf("block.x %d block.y %d block.z %d\n",block.x,
block.y, block.z);

// check grid and block dimension from device side
checkIndex <<<grid, block>>> ();
```

Check Grid and Block Indices

- In the kernel function, each thread prints out its own thread index, block index, block dimension, and grid dimension

```
__global__ void checkIndex(void) {
  printf("threadIdx:(%d, %d, %d) blockIdx:(%d, %d, %d)
blockDim:(%d, %d, %d) "
  "gridDim:(%d, %d, %d)\n", threadIdx.x, threadIdx.y,
threadIdx.z,
  blockIdx.x, blockIdx.y, blockIdx.z, blockDim.x,
blockDim.y, blockDim.z, gridDim.x,gridDim.y,gridDim.z);
}
```

checkDimension.cu



Access Grid/Block Variables From Host and Device Side

- It is important to distinguish between the host and device access of grid and block variables.

- On the host side, using a variable declared as block:

  block.x, block.y, and block.z

- On the device side, using pre-initialized, built-in block size variable:

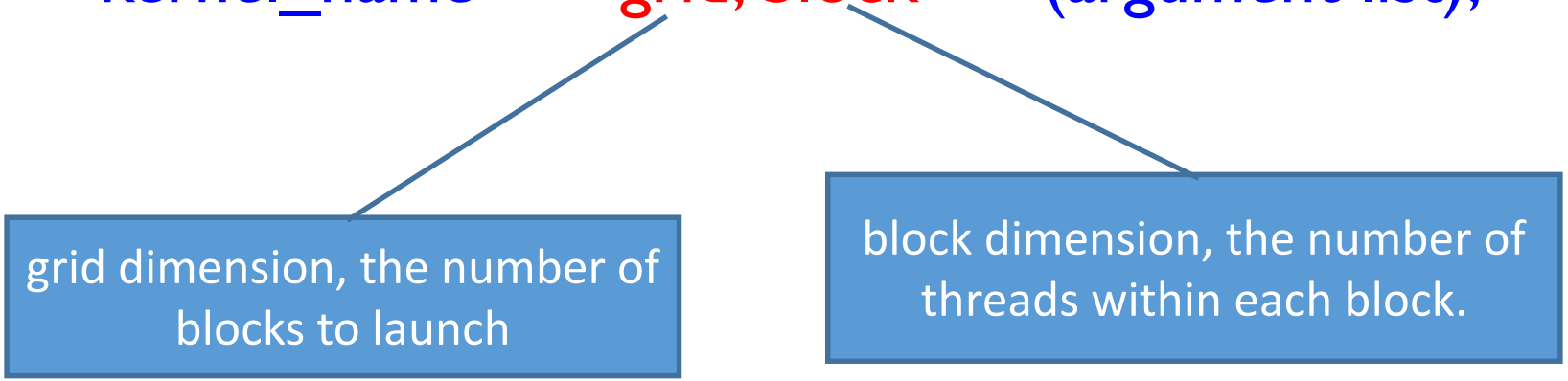
  blockDim.x, blockDim.y, and blockDim.z

1.4 Launching a CUDA Kernel

Launching a CUDA Kernel

- A CUDA kernel call is a direct extension to C function syntax that adds a kernel's execution configuration:

```
kernel_name <<<grid, block>>>(argument list);
```

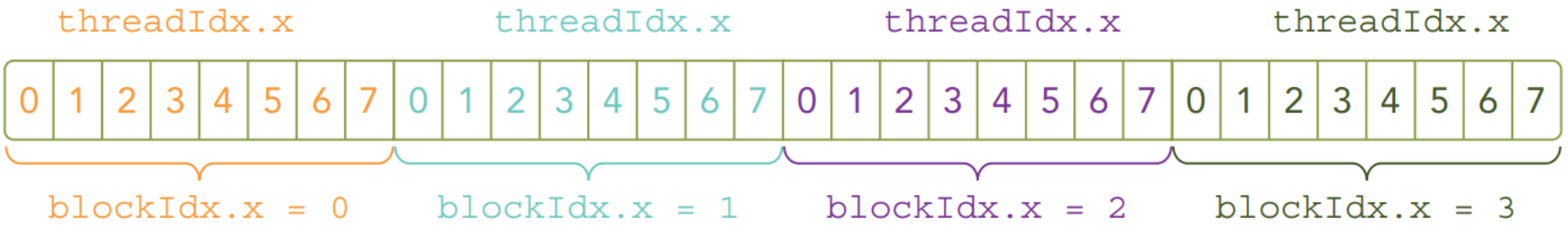


- By specifying the grid and block dimensions, you configure:
  - The total number of threads for a kernel
  - The layout of the threads you want to employ for a kernel

Example

- Suppose you have 32 data elements for a calculation.
- You can group 8 elements into each block, and launch four blocks as follows:

```
kernel_name<<<4, 8>>>(argument list);
```



- use built-in variables blockIdx.x and threadIdx.x to:
  - Identify a unique thread in the grid.
  - Establish a mapping between threads and data elements.

Asynchronous Behaviors

- Unlike a C function call, all CUDA kernel launches are **asynchronous.**
  - Control returns to the CPU **immediately** after the CUDA kernel is invoked

- You can call the following function to force the host application to wait for all kernels to complete.

```
cudaDeviceSynchronize(void);
```

1.5 Writing Your Kernel

Writing Your Kernel

- **A kernel function is the code to be executed on the device side.**
  - In a kernel function, you define the computation for a single thread, and the data access for that thread.
  - When the kernel is called, many different CUDA threads perform the same computation in parallel.

- A kernel is defined using __global__ declaration specification as shown:

```
__global__ void kernel_name(argument list);
```

Function Type Qualifiers

Function type qualifiers specify:

(1) whether a function executes on the host or on the device and

(2) whether it is callable from the host or from the device.

QUALIFIERS	EXECUTION	CALLABLE	NOTES
`__global__`	Executed on the device	Callable from the host Callable from the device for devices of compute capability 3	Must have a `void` return type
`__device__`	Executed on the device	Callable from the device only	
`__host__`	Executed on the host	Callable from the host only	Can be omitted

CUDA Kernels are Functions with Restrictions

The following restrictions apply for all kernels:

- Access to device memory only
- Must have void return type
- No support for a variable number of arguments
- No support for static variables
- No support for function pointers
- Exhibit an asynchronous behavior

A Simple Example: sumArrary

```
void sumArraysOnHost(float *A, float *B, float *C, const int
N) {
        for (int i = 0; i < N; i++)
                C[i] = A[i] + B[i];
}
```



```
__global__ void sumArraysOnGPU(float *A, float *B, float *C)
{
        int i = threadIdx.x;
        C[i] = A[i] + B[i];
}
```

Difference: **the loop is missing!**

- Supposing a vector with the length of 32 elements, you can invoke the kernel with 32 threads as follows:

  sumArraysOnGPU<<<1,32>>>(float *A, float *B, float *C);

1.6 Verifying Your Kernel

Verifying Your Kernel

- Two very basic means to verify your kernel code.
  - 1) use printf in your kernel for Fermi and later generation devices.
  - 2) set the execution configuration to <<<1,1>>>, so you force the kernel to run with only one block and one thread.

```
void checkResult(float *hostRef, float *gpuRef, const int N) {
  double epsilon = 1.0E-8;
  int match = 1;

  for (int i = 0; i < N; i++) {
    if (abs(hostRef[i] - gpuRef[i]) > epsilon) {
      match = 0;
      printf("Arrays do not match!\n");
      printf("host %5.2f gpu %5.2f at current %d\n",
        hostRef[i], gpuRef[i], i);
      break;
    }
  }
}
```

1.7 Handling Errors

Handling Errors

- Defining an error-handling macro to wrap all CUDA API calls simplifies the error checking process:

```
#define CHECK(call)                                    \
{                                                      \
  const cudaError_t error = call;                      \
  if (error != cudaSuccess)                            \
  {                                                    \
    printf("Error: %s:%d, ", __FILE__, __LINE__); \
    printf( “code:%d, reason: %s\n” , error,
cudaGetErrorString(error));                            \
    exit(1);                                           \
  }                                                    \
}
```

- Example:

CHECK(cudaMemcpy(d_C, gpuRef, nBytes, cudaMemcpyHostToDevice));

1.8 Compiling and Executing

Compiling and Executing

- GPU-based vector summation (sumArraysOnGPU-small-case.cu)

```
// malloc device global memory
float *d_A, *d_B, *d_C;
cudaMalloc((float**)&d_A, nBytes);
cudaMalloc((float**)&d_B, nBytes);
cudaMalloc((float**)&d_C, nBytes);
// transfer data from host to device
cudaMemcpy(d_A, h_A, nBytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, nBytes, cudaMemcpyHostToDevice);
// invoke kernel at host side
dim3 block (nElem);
dim3 grid (nElem/block.x);
sumArraysOnGPU<<< grid, block >>>(d_A, d_B, d_C);
printf("Execution configuration <<<%d, %d>>>\n",grid.x,block.x);
// copy kernel result back to host side
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyDeviceToHost);
// add vector at host side for result checks
sumArraysOnHost(h_A, h_B, hostRef, nElem);
// check device results
checkResult(hostRef, gpuRef, nElem);
```

Compiling and Executing

- For the general case, you can calculate the unique index of global data access for a given thread based on the 1D grid and block information as follows:

```
__global__ void sumArraysOnGPU(float *A, float *B, float *C)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    C[i] = A[i] + B[i];
}
```



sumArraysOnGPU-small-case.cu

2. Timing Your Kernel

2.1 Timing with CPU Timer

Timing with CPU Timer

- A CPU timer can be created by using the gettimeofday system call to get the system's wall-clock time

```
double cpuSecond() {
    struct timeval tp;
    gettimeofday(&tp,NULL);
    return ((double)tp.tv_sec + (double)tp.tv_usec*1.e-6);
}
```

- You can measure your kernel with cpuSecond:

```
double iStart = cpuSecond();
kernel_name<<<grid, block>>>(argument list);
cudaDeviceSynchronize();  //wait for all GPU threads to complete.
double iElaps = cpuSecond() - iStart;
```

Measuring vector summation kernel

- Measuring the vector summation kernel (sumArraysOnGPU-timer.cu)

```
$ nvcc sumArraysOnGPU-timer.cu -o sumArraysOnGPU-timer
$ ./sumArraysOnGPU-timer
./sumArraysOnGPU-timer Starting...
Using Device 0:Tesla M2070
Vector size 16777216
sumArraysOnGPU <<<16384, 1024>>> Time elapsed 0.002456 sec
Arrays match.
```



sumArraysOnGPU-timer.cu

Know Your Limitations

- A key concept to understand while tweaking the execution configuration is the limitations on grid and block dimensions.
- The maximum size at each level of the thread hierarchy is device dependent.
  - For Fermi devices, the maximum number of threads per block is 1,024
  - the maximum grid dimension for each x, y, and z dimension is 65,535.

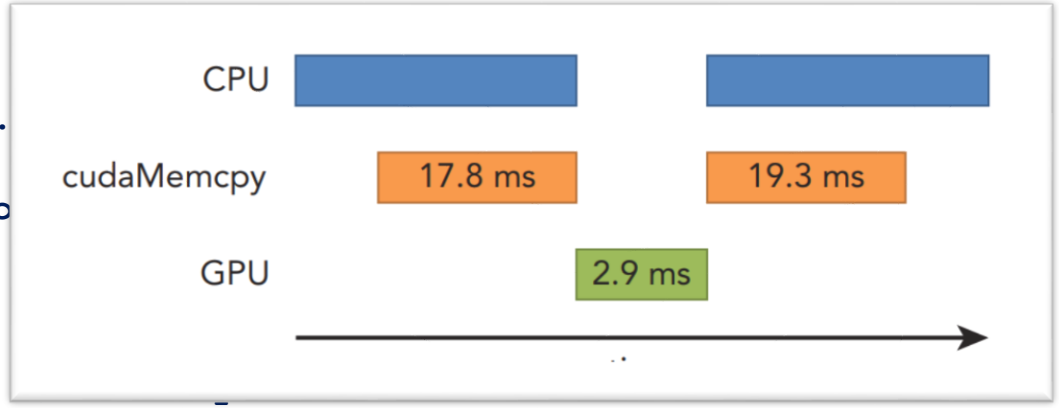
```
sumArraysOnGPU <<<65536, 256>>> Time elapsed 0.000183 sec
Error: sumArraysOnGPU-timer.cu:153, code:9, reason: invalid configuration argument
```

2.2 Timing with nvprof

Timing with nvprof

- **nvprof:** a command-line profiling tool of CUDA
  - collect timeline information of you app's CPU and GPU activity, including kernel execution, memory transfers, and CUDA API calls.

$ nvprof [nvprof_args] <application> [application_args]




```
$ nvprof ./sumArraysOnGPU-timer
./sumArraysOnGPU-timer Starting..
Using Device 0: Tesla M2070
==17770== NVPROF is profiling pro
Vector size 16777216
sumArraysOnGPU <<<16384, 1024>>>
Arrays match.
==17770== Profiling application:
==17770== Profiling result:
Time(%)  Time      Calls   Avg       Min       Max       Name
70.35%   52.667ms  3       17.556ms  17.415ms  17.800ms  [CUDA memcpy HtoD]
25.77%   19.291ms  1       19.291ms  19.291ms  19.291ms  [CUDA memcpy DtoH]
3.88%    2.9024ms  1       2.9024ms  2.9024ms  2.9024ms  sumArraysOnGPU
(float*, float*, int)
```

Theoretical Performance Limits

- Determine if your application is limited by arithmetic or by memory bandwidth.
- Example： **Telsa K10**
  - Tesla K10 Peak Single Precision FLOPS:
    - 745 MHz core clock * 2 GPUs/board * (8 multiprocessors * 192 fp32 cores/multiprocessor) * 2 ops/cycle = 4.58 TFLOPS
  - Tesla K10 Peak Memory Bandwidth:
    - 2 GPUs/board * 256 bit * 2500 MHz mem-clock * 2 DDR / 8 bits/byte = 320 GB/s
  - Ratio of instruction:bytes:
    - 4.58 TFLOPS / 320 GB/s yields 13.6 instructions:1 byte
- If issuing more than 13.6 instructions for every byte accessed, then bound by arithmetic performance.
  - Most HPC workloads are bound by memory bandwidth

3. Organizing Parallel Threads

Organizing Parallel Threads

- If you properly organize threads using the right grid and block size, it can make a big impact on kernel performance

- Learn more about grid and block heuristics using the following layouts for matrix addition:
  - 2D grid with 2D blocks
  - 1D grid with 1D blocks
  - 2D grid with 1D blocks

3.1 Indexing Matrices with Blocks and Threads

Indexing Matrices with Blocks/ Threads

For a thread, **accessing the assigned data from global memory** using block and thread index is the first issue you need to solve.

Typically, there are three kinds of indices for a 2D case:

- Thread and block index: threadIdx, blockIdx
- Coordinate of a given point in the matrix: ix, iy
- Offset in linear global memory: idx

We need to obtain the offset in global memory from the block and thread index

Two Steps:

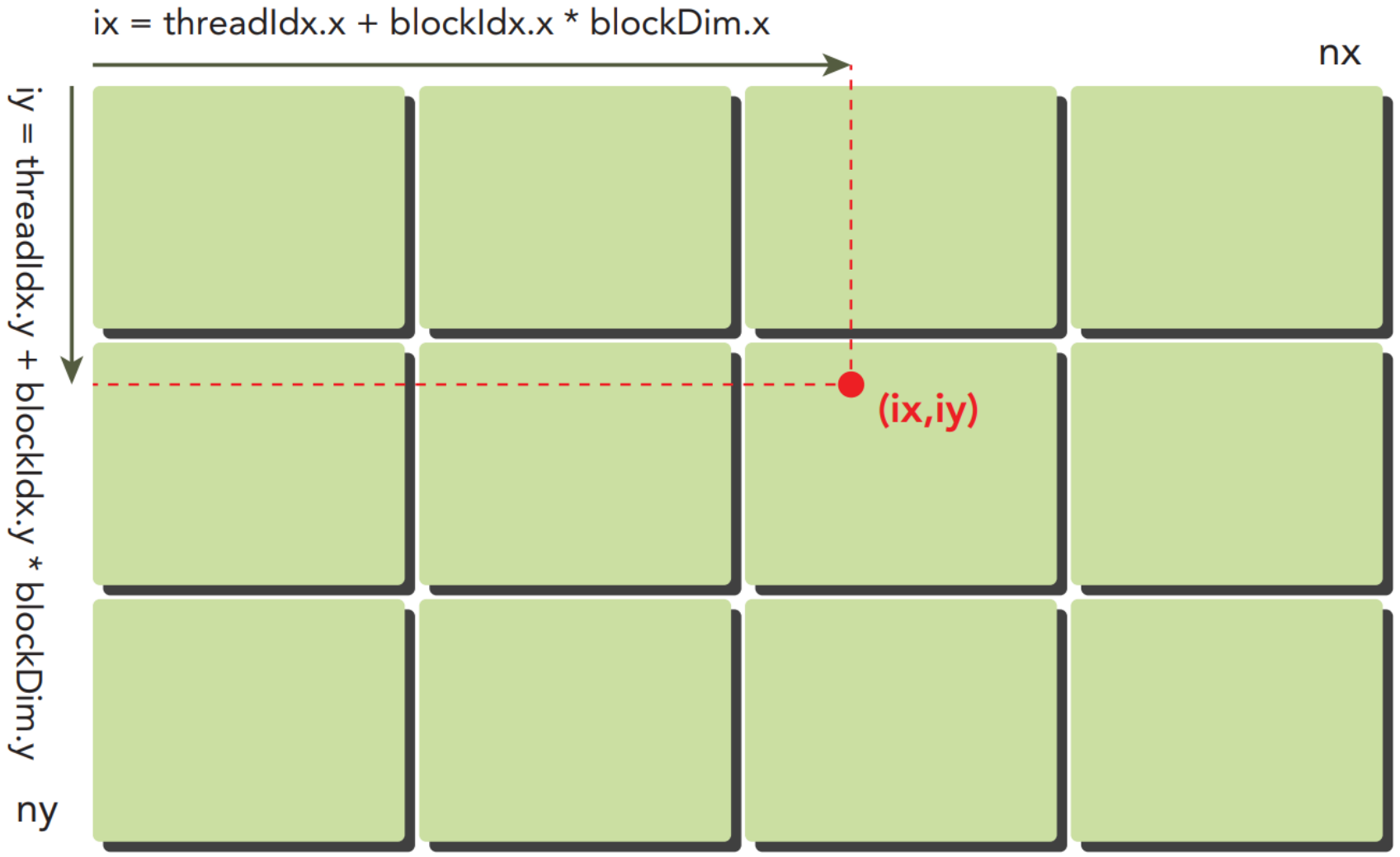
1) map thread and block index to the coordinate of a matrix by:

```
ix = threadIdx.x + blockIdx.x * blockDim.x
iy = threadIdx.y + blockIdx.y * blockDim.y
```

2) map a matrix coordinate to a global memory location/index by:

```
idx = iy * nx + ix
```

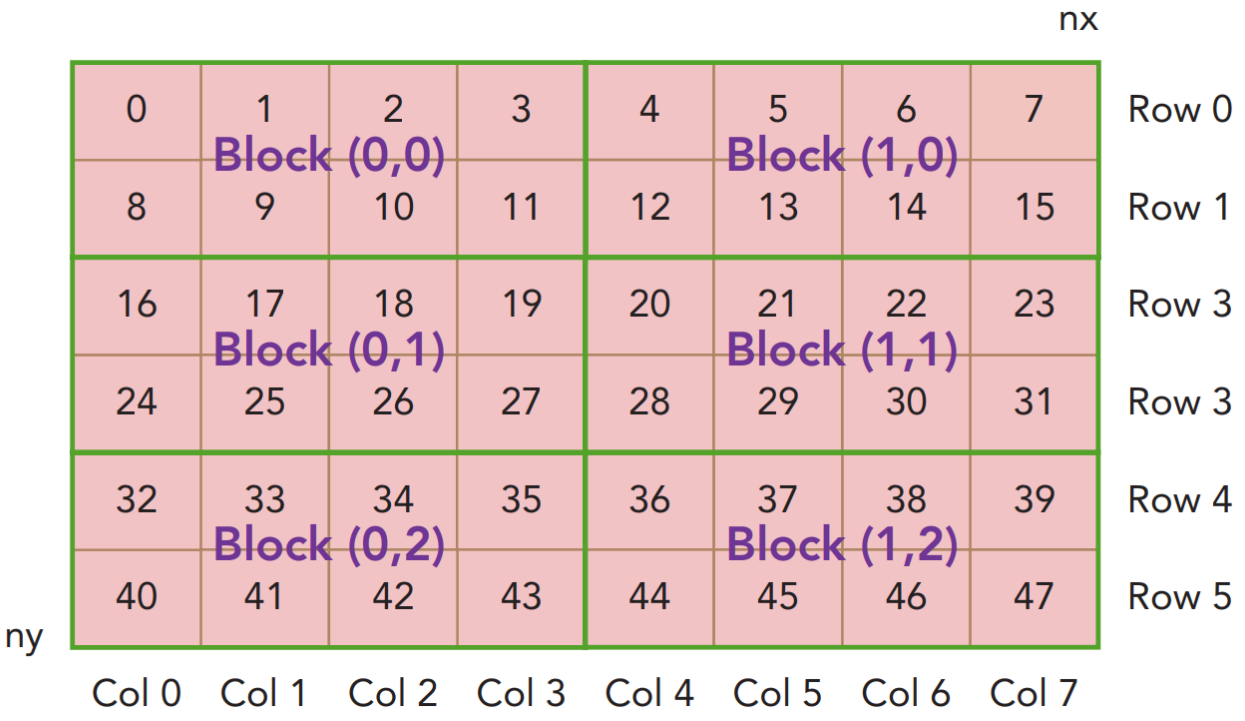
Relationship among block and thread indices, matrix coordinates, and linear global memory indices.



matrix coordinate: (ix,iy)
global linear memory index: idx = iy*nx + ix

Check Block and Thread Index

- printThreadInfo prints out the following information about each thread:
  - Thread index
  - Block index
  - Matrix coordinate
  - Global linear memory offset
  - Value of corresponding elements





checkThreadIndex.cu

3.2 Summing Matrices with a 2D Grid and 2D Blocks

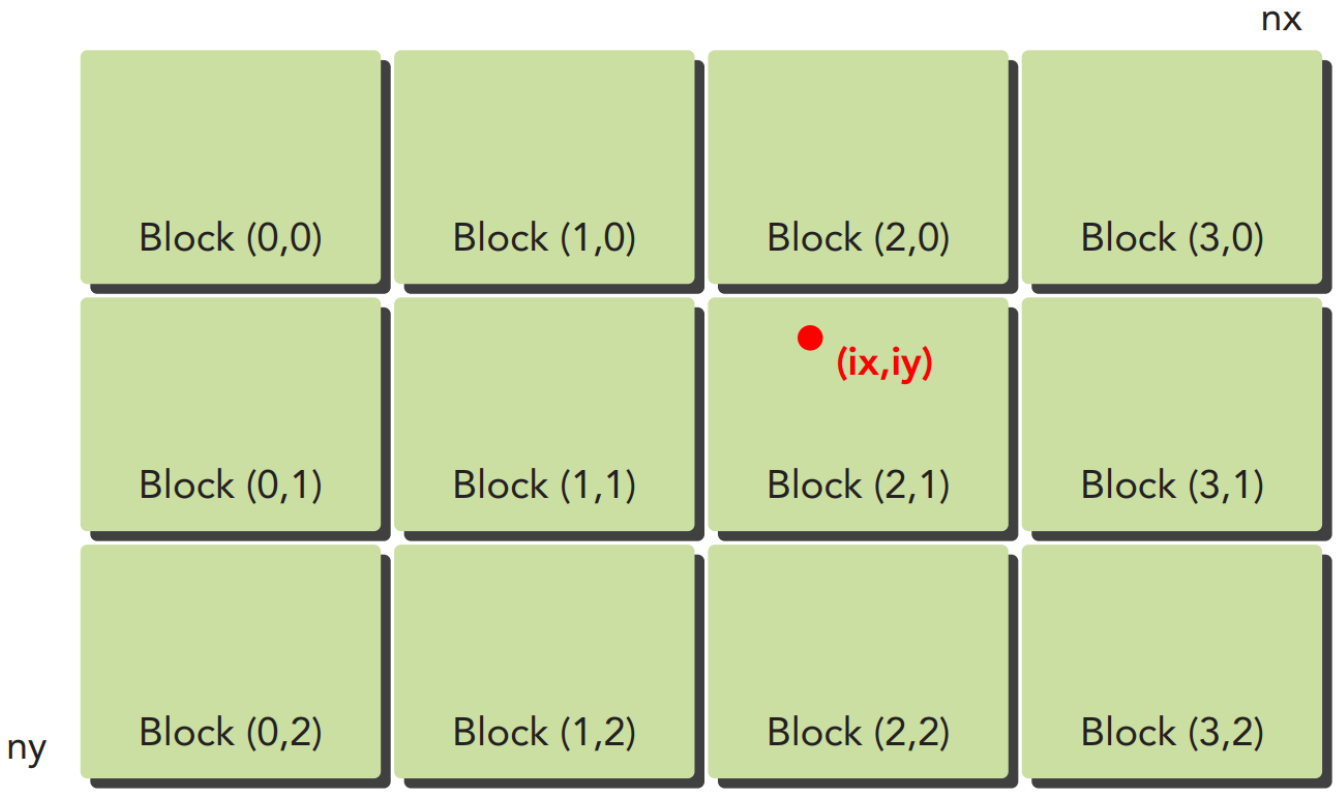
Summing Matrices with a 2D Grid and 2D Blocks

- Write a matrix addition kernel that uses a 2D grid with 2D blocks.
- Create a new kernel to sum the matrix with a 2D thread block:

```
__global__ void sumMatrixOnGPU2D(float *MatA, float
*MatB, float *MatC, int nx, int ny) {
  unsigned int ix = threadIdx.x + blockIdx.x * blockDim.x;
  unsigned int iy = threadIdx.y + blockIdx.y * blockDim.y;
  unsigned int idx = iy*nx + ix;
  if (ix < nx && iy < ny)
    MatC[idx] = MatA[idx] + MatB[idx];
}
```

Summing Matrices with a 2D Grid and 2D Blocks

- The key to this kernel is the step mapping each thread from **its thread index** to **global linear memory index**



Summing Matrices with a 2D Grid and 2D Blocks

- The matrix size can be set to **16,384 elements** in each dimension as follows:

```
int nx = 1<<14;
int ny = 1<<14;
```

- Then, the kernel execution configuration can be set to use a **2D grid and 2D blocks** as follows:

```
int dimx = 32;
int dimy = 32;

dim3 block(dimx, dimy);
dim3 grid((nx + block.x - 1) / block.x,
          (ny + block.y - 1) / block.y);
```

Summing Matrices with a 2D Grid and 2D Blocks

- Increasing the number of blocks **does not necessarily** always **increase** the performance of the kernel
  - More blocks, more parallelism?

KERNEL CONFIGURATION	KERNEL ELAPSED TIME	BLOCK NUMBER
(32,32)	0.060323 sec	512 x 512
(32,16) Block dims	0.038041 sec **Fastest!**	512 x 1024
(16,16)	0.045535 sec	1024 x 1024 **Highest!**

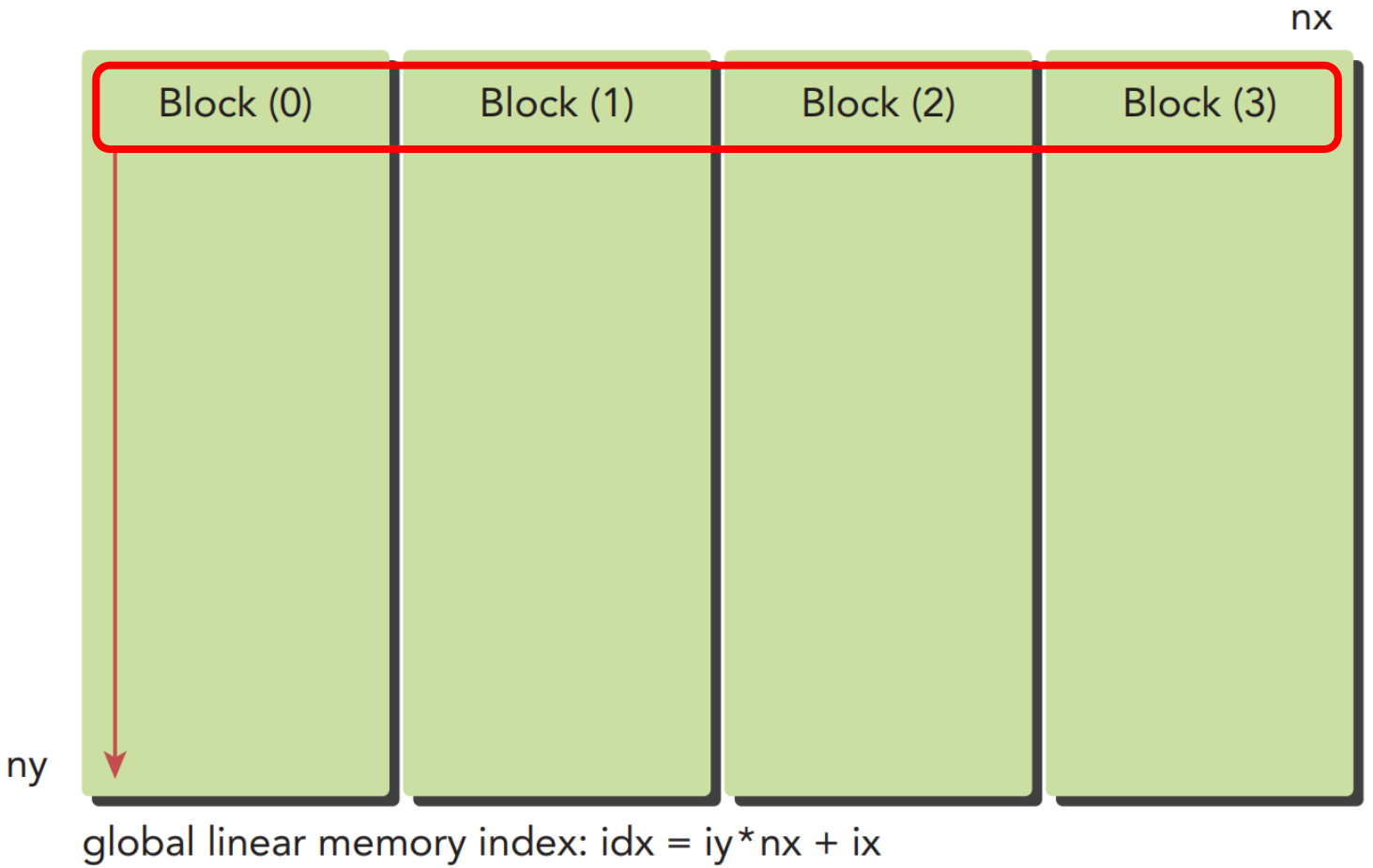


sumMatrixOnGPU-2D-grid-2D-block.cu

3.3 Summing Matrices with a 1D Grid and 1D Blocks

Summing Matrices with a 1D Grid and 1D Blocks

- To use a **1D grid with 1D blocks**, you need to write a new kernel in which each thread processes **`ny`** data elements



Summing Matrices with a 1D Grid and 1D Blocks

- Because a 1D block layout is used for kernel launch, **only threadIdx.x is useful** and a **loop** is used to handle **ny** elements in each thread.

```
__global__ void sumMatrixOnGPU1D(float *MatA, float *MatB,
float *MatC, int nx, int ny) {
      unsigned int ix = threadIdx.x + blockIdx.x * blockDim.x;
      if (ix < nx ) {
        for (int iy=0; iy<ny; iy++) {
              int idx = iy*nx + ix;
              MatC[idx] = MatA[idx] + MatB[idx];
        }
      }
    }
```

- Next, set the 1D grid and block configuration as follows:

```
dim3 block(32,1);
dim3 grid((nx+block.x-1)/block.x, 1);
```

- Invoke the kernel with the 1D configuration as follows:

```
sumMatrixOnGPU1D <<< grid, block >>>(d_MatA, d_MatB,
d_MatC, nx, ny)
```

Summing Matrices with a 1D Grid and 1D Blocks

```
$ nvcc sumMatrixOnGPU-1D-grid-1D-block.cu -o matrix1D
$ ./matrix1D
sumMatrixOnGPU1D <<<(512,1), (32,1)>>> elapsed 0.061352 sec
```

- Next, increase the block size as follows:

```
dim3 block(128,1);
dim3 grid((nx+block.x-1)/block.x,1);
sumMatrixOnGPU1D <<<(128,1),(128,1)>>> elapsed 0.044701 sec
```

- The result shows that performance is basically the same compared to the result using a 2D grid and block (32 x 32) configuration.

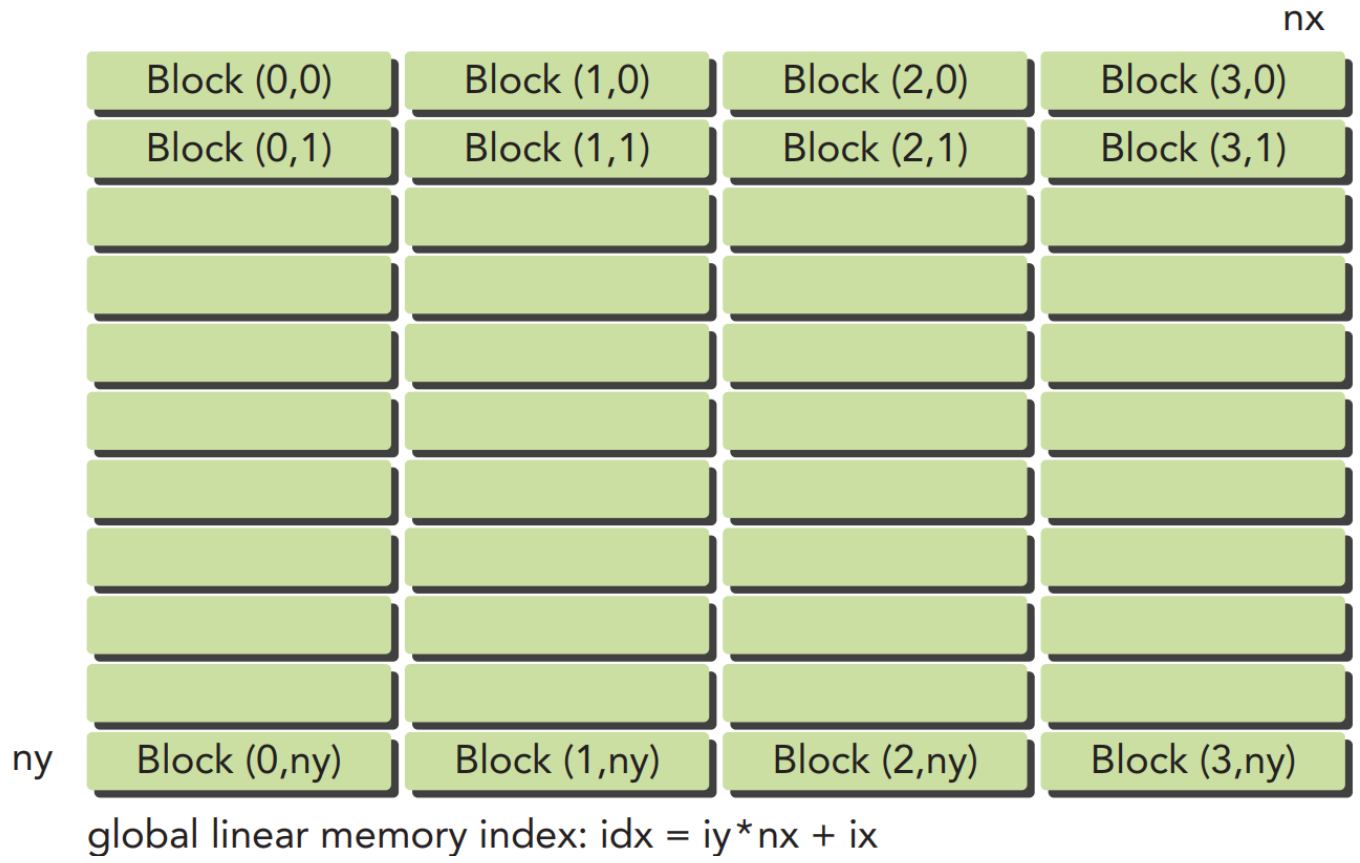


sumMatrixOnGPU-1D-grid-1D-block.cu

3.4 Summing Matrices with a 2D Grid and 1D Blocks

Summing Matrices with a 2D Grid and 1D Blocks

- When using a 2D grid that contains 1D blocks, each thread takes care of only one data element and the second dimension of grid equals **ny**



Summing Matrices with a 2D Grid and 1D Blocks

- This can be viewed as a **special case of a 2D grid with 2D blocks**, where the second dimension of the block is 1.
- The mapping from block and thread index to the matrix coordinate becomes:

```
ix = threadIdx.x + blockIdx.x * blockDim.x;
iy = blockIdx.y;
```

- The new kernel is shown here:

```
__global__ void sumMatrixOnGPUMix(float *MatA, float *MatB,
float *MatC, int nx, int ny) {
    unsigned int ix = threadIdx.x + blockIdx.x * blockDim.x;
    unsigned int iy = blockIdx.y;
    unsigned int idx = iy*nx + ix;
    if (ix < nx && iy < ny)
        MatC[idx] = MatA[idx] + MatB[idx];
    }
```

Summing Matrices with a 2D Grid and 1D Blocks

Results Comparison of Different Kernel Implementations

KERNEL	EXECUTION CONFIGURE	TIME ELAPSED
`sumMatrixOnGPU2D`	(512,1024), (32,16)	0.038041
`sumMatrixOnGPU1D`	(128,1), (128,1)	0.044701
`sumMatrixOnGPUMix`	(64,16384), (256,1)	0.030765

From the matrix addition examples, you can see several things:

- Changing execution configurations affects performance.
- A naive kernel implementation does not generally yield the best performance.
- For a given kernel, trying different grid and block dimensions may yield better performance

4. Managing Devices

4.1 Using the Runtime API to Query GPU Information

Using the Runtime API to Query GPU Information

- Many functions are available in the CUDA runtime API to help you manage devices.
  - The properties of the GPU device are returned in the cudaDeviceProp structure

cudaGetDeviceProperties(cudaDeviceProp* prop, int device);

```
./checkDeviceInfor Starting...
Detected 2 CUDA Capable device(s)
Device 0: "Tesla M2070"
  CUDA Driver Version / Runtime Version          5.5 / 5.5
  CUDA Capability Major/Minor version number:    2.0
  Total amount of global memory:                 5.25 MBytes (5636554752 bytes)
  GPU Clock rate:                                1147 MHz (1.15 GHz)
  Memory Clock rate:                             1566 Mhz
  Memory Bus Width:                              384-bit
  L2 Cache Size:                                 786432 bytes
  Max Texture Dimension Size (x,y,z) 1D=(65536), 2D=(65536,65535), 3D=(2048,2048,2048)
  Max Layered Texture Size (dim) x layers 1D=(16384) x 2048, 2D=(16384,16384) x 2048
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total number of registers available per block: 32768
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  1536
  Maximum number of threads per block:           1024
  Maximum sizes of each dimension of a block:    1024 x 1024 x 64
  Maximum sizes of each dimension of a grid:     65535 x 65535 x 65535
  Maximum memory pitch:                          2147483647 bytes
```



checkDeviceInfor.cu

4.2 Determining the Best GPU

Determining the Best GPU

- Some systems support **multiple GPUs**.
  - One way to identify the most computationally capable GPU is by the number of multiprocessors it contains.

```
int numDevices = 0;
cudaGetDeviceCount(&numDevices);
if (numDevices > 1) {
   int maxMultiprocessors = 0, maxDevice = 0;
   for (int device=0; device<numDevices; device++) {
      cudaDeviceProp props;
      cudaGetDeviceProperties(&props, device);
      if (maxMultiprocessors < props.multiProcessorCount) {
         maxMultiprocessors = props.multiProcessorCount;
         maxDevice = device;
      }
   }
   cudaSetDevice(maxDevice);
}
```

4.3 Using nvidia-smi to Query GPU Information

Using nvidia-smi to Query GPU Info.

- The command-line tool **`nvidia-smi`** allows you to query and modify device state.

```
$ nvidia-smi -L
GPU 0: Tesla M2070 (UUID: GPU-68df8aec-e85c-9934-2b81-0c9e689a43a7)
GPU 1: Tesla M2070 (UUID: GPU-382f23c1-5160-01e2-3291-ff9628930b70)

$ nvidia-smi -q -i 0

$ nvidia-smi -q -i 0 -d MEMORY | tail -n 5
Memory Usage
Total : 5375 MB
Used : 9 MB
Free : 5366 MB

$ nvidia-smi -q -i 0 -d UTILIZATION | tail -n 4
Utilization
Gpu : 0 %
Memory : 0 %
```

determine how many GPUs are installed on a system

report details about GPU 0

display device memory information only

display device utilization information only

4.4 Setting Devices at Runtime

Setting Devices at Runtime

- For a system that has `N` GPUs installed, the device IDs reported by nvidia-smi are **labeled `0` through `N-1`**.
- Using environment variable **CUDA_VISIBLE_DEVICES**, it is possible to specify which GPUs to use **at runtime**.

Example:

- Set environment variable **CUDA_VISIBLE_DEVICES = 2** at runtime.
- The nvidia driver masks off the other GPUs so that device 2 appears to your application as device 0.

Summary

- Compared with parallel programming in C, the **thread hierarchy** in CUDA programming is a distinguishing feature.

- For a given problem, you have **several options** to implement the kernel and different configurations to execute the kernel.
  - Therefore, learning **how to organize threads** is one of the central practices of CUDA programming.

- **Grids and blocks** represent a logical view of the thread layout for kernel execution.